Security Incident Information System: A Relational Approach*

Kousar Malekinasab
University of Victoria
Canada
kovvy@uvic.ca

Hossein Fani
University of Victoria
Canada
hfani@uvic.ca

ABSTRACT

This is a sample proposed method. Adapt its sections, relations, and queries to your ER diagram. The supplied figures are teaching illustrations, not implementation evidence. Security incident information connects actors, actions, assets, evidence, and impacts. This sample maps them to relations with surrogate identifiers, candidate keys, and foreign-key constraints, and illustrates retrieval and modification using SQLite. Provide code, setup instructions, artifacts, a license, and AI disclosure at `[Insert project repository URL]`.

KEYWORDS

Security incidents, relational data modeling, SQLite, SQL, data integrity

1 PROBLEM DEFINITION

Our goal is to let analysts retrieve incident timelines, affected assets, and supporting evidence. Flat actor and asset lists lose action–target associations. We therefore link each action to one incident, one asset, and an optional actor. Cross-incident technique queries support comparison. Temporal order does not establish causation. State your domain, intended users, and the questions your database must answer. Explain how your model addresses those questions.

2 PROPOSED RELATIONAL MODEL

2.1 Incident and Core Entity Relations

`Incident` records a reference, title, occurrence time, severity, and status. `Actor`, `Asset`, and `Technique` describe reusable entities. Business identifiers such as `Reference` and `AssetCode` are mandatory candidate keys; names are not assumed unique. Every relation has an autogenerated integer primary key named `Id`. The complete relation list appears on the final page.

2.2 Action, Observation, and Impact Relations

`Action` records one target-specific event and references one incident, one asset, and optionally one actor. Unknown attribution is represented by NULL, while the incident and asset references are mandatory. `Observation` stores a source, collection time, and evidence detail for one action; several observations can support the same action. `Impact` connects an incident and asset to an impact dimension. For this sample, one description summarizes each incident–asset–dimension combination. This is a modeling assumption, not a universal security rule.

2.3 Relationship Mapping and Integrity

The action–technique many-to-many relationship becomes `ActionTechnique`, with a unique pair of foreign keys. One-to-many relationships use foreign keys on the many side. Business candidate keys use both UNIQUE and NOT NULL. Allowed severity, status, and impact values use CHECK constraints. Foreign-key enforcement must be enabled on every SQLite connection. Referenced records cannot be deleted while dependent rows remain; no cascade is assumed. Justify cardinalities, optionality, and candidate keys using your ER diagram. Explain multivalued attributes and IS-A mappings if present.

3 RELATIONAL OPERATIONS

3.1 Creating and Populating Relations

The following excerpt illustrates the surrogate key, a business candidate key, and domain constraints. The repository must contain DDL for *all* relations, created before dependent records are inserted.

```
PRAGMA foreign_keys = ON;
CREATE TABLE Incident (
 Id INTEGER PRIMARY KEY AUTOINCREMENT,
 Reference TEXT NOT NULL,
 Title TEXT NOT NULL,
 OccurredAt TEXT NOT NULL,
 Severity TEXT NOT NULL CHECK
   (Severity IN ('Low','Medium','High','Critical')),
 Status TEXT NOT NULL CHECK
   (Status IN ('Open','Investigating','Closed')),
 CONSTRAINT UK_Incident_Reference UNIQUE(Reference)
);
```

For `Action.IncidentId`, the corresponding constraint is:

```
CONSTRAINT FK_Action_IncidentId_2_Incident_Id
 FOREIGN KEY (IncidentId) REFERENCES Incident(Id)
```

Seed incident `INC-001`, asset `SRV-001`, and action `EVT-001`. Insert parents first; resolve generated identifiers by business keys.

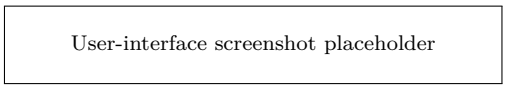


Figure 1: Supplied UI placeholder; replace with screenshots of actual operations.

*Link to Github or an online repository: `[Insert project repository URL]`.

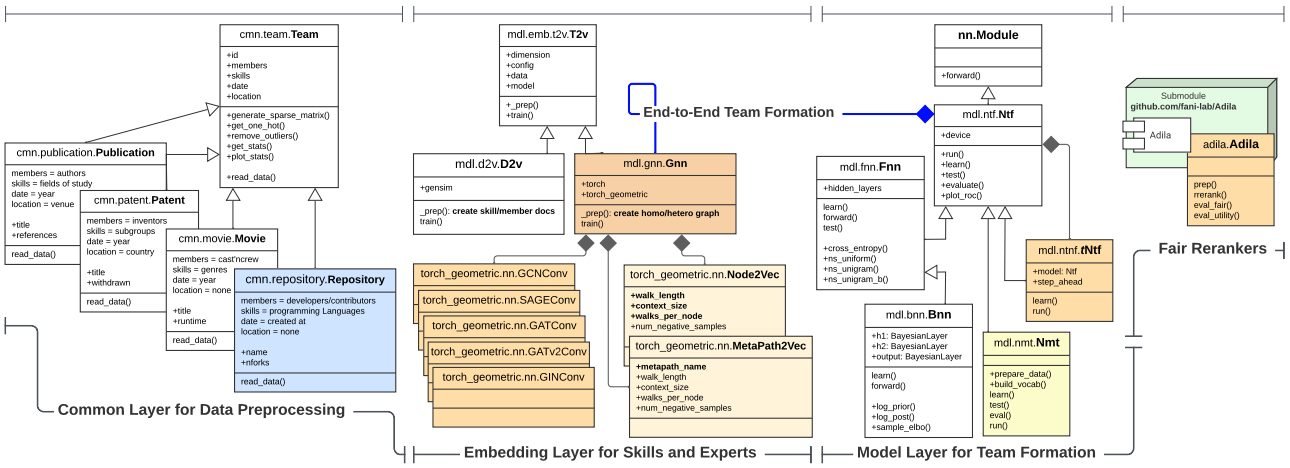


Figure 2: Architecture example supplied by the instructor (team-formation domain), preserved as a template illustration. Replace it with your own system architecture.

3.2 SELECT: Incident Timeline

```
SELECT I.Reference, A.EventReference,
       A.OccurredAt, S.AssetCode, R.Name
FROM Incident I
JOIN Action A ON A.IncidentId = I.Id
JOIN Asset S ON S.Id = A.AssetId
LEFT JOIN Actor R ON R.Id = A.ActorId
WHERE I.Reference = 'INC-001'
ORDER BY A.OccurredAt, A.Id;
```

The left join retains actions with an unknown actor. With the illustrative records above, the result associates `EVT-001` with `SRV-001`. This is an expected example, not a measured execution result. Evidence retrieval joins `Observation` to `Action` and then `Incident`. A cross-incident technique query illustrates aggregation:

```
SELECT T.TechniqueCode,
       COUNT(DISTINCT A.IncidentId) AS IncidentCount
FROM Technique T
JOIN ActionTechnique X ON X.TechniqueId = T.Id
JOIN Action A ON A.Id = X.ActionId
GROUP BY T.Id, T.TechniqueCode;
```

DISTINCT avoids counting an incident repeatedly when several actions use the same technique. Prioritize SELECT queries that answer your actual information needs; explain joins, filters, aggregation, and meaningful outputs.

3.3 INSERT, UPDATE, and DELETE

```
INSERT INTO Incident
 (Reference,Title,OccurredAt,Severity,Status)
VALUES ('INC-DEMO','Synthetic example',
 '2026-10-01T10:00:00Z','Low','Open');
UPDATE Incident SET Status = 'Closed'
WHERE Reference = 'INC-DEMO';
DELETE FROM Incident WHERE Reference = 'INC-DEMO';
```

These statements illustrate a temporary incident without dependent records. In an application, bind user inputs as parameters and commit successful transactions or roll back failures. Reject missing values, invalid timestamps, and unsupported categories. Explain how dependent records affect deletion.

4 ARCHITECTURE AND USER INTERFACE

The incident application separates forms and search controls, validation and transaction logic, and parameterized SQLite access. Results and errors return to the interface. Users can enter, search, update, and delete incidents without writing SQL; updates preserve record identity.

Explain actual layers and information flow (Figure 2); replace the UI placeholder (Figure 1) with screenshots of operations and results.

5 REPRODUCIBILITY AND EVALUATION

Provide separate DDL, seed data, queries, application code, and a quickstart. Document dependencies, foreign-key configuration, license, and reproduction steps. Test duplicate keys, invalid references, optional attribution, and restricted deletion; report observed results. Link the evaluator-accessible repository and document individual contributions and critical evaluation of AI assistance. This sample used ChatGPT [1]; students must report their own evaluation.

REFERENCES

[1] OpenAI. 2026. ChatGPT. AI assistance for sample text, relational design, and SQL drafting, 20 October 2026.

RELATIONS AND CONSTRAINTS: ILLUSTRATIVE ER-TO-RELATIONAL MAPPING

Replace this list with *all* relations in your project. For each, identify its autogenerated surrogate key, business candidate keys, foreign keys, mandatory/optional attributes, and relevant domain constraints.

An underlined Id denotes the autogenerated INTEGER primary key. CK below lists additional business candidate keys, each enforced by UNIQUE and NOT NULL; {Id} is also a candidate key in every relation. All non-key attributes are mandatory unless explicitly marked optional. All foreign keys reference the indicated relation's Id, with no cascading deletion.

(1) Incident<u>Id</u>, Reference, Title, OccurredAt, Severity, Status.
CK: {Reference}, mandatory. Severity ∈ {Low, Medium, High, Critical}; Status ∈ {Open, Investigating, Closed}.

(2) Actor<u>Id</u>, ActorCode, Name, Category.
CK: {ActorCode}, mandatory. Category is mandatory text. Name need not be unique.

(3) Asset<u>Id</u>, AssetCode, Name, Kind.
CK: {AssetCode}, mandatory. Kind is mandatory text.

(4) Technique<u>Id</u>, TechniqueCode, Name.
CK: {TechniqueCode}, mandatory. Codes are unique within the chosen vocabulary.

(5) Action<u>Id</u>, EventReference, IncidentId, ActorId, AssetId, OccurredAt, Description.
CK: {EventReference}, mandatory.
FK_1: IncidentId → Incident.Id, mandatory; FK_2: ActorId → Actor.Id, optional; FK_3: AssetId → Asset.Id, mandatory.
Each action belongs to exactly one incident and targets exactly one asset. Multiple target assets require separate target-specific actions in this sample.

(6) ActionTechnique<u>Id</u>, ActionId, TechniqueId.
CK: {ActionId, TechniqueId}, mandatory.
FK_1: ActionId → Action.Id, mandatory; FK_2: TechniqueId → Technique.Id, mandatory.
This associative relation implements the many-to-many action–technique relationship.

(7) Observation<u>Id</u>, EvidenceReference, ActionId, ObservedAt, Source, Detail.
CK: {EvidenceReference}, mandatory. FK: ActionId → Action.Id, mandatory.
Each observation supports exactly one action; an action may have zero or more observations.

(8) Impact<u>Id</u>, IncidentId, AssetId, Dimension, Description.
CK: {IncidentId, AssetId, Dimension}, mandatory.
FK_1: IncidentId → Incident.Id, mandatory; FK_2: AssetId → Asset.Id, mandatory.
Dimension ∈ {Confidentiality, Integrity, Availability}. At most one summary is stored for each incident–asset–dimension combination.

Mapping assumptions. Incidents, actors, assets, and techniques may exist before their related rows are entered; hence parent-side participation is optional. Every action must reference an existing incident and asset. Attribution may be unknown. An impact may be recorded before the responsible action is known. Occurrence and observation times are UTC text in a fixed ISO-style format; input validation must enforce the format. Ordering timestamps does not establish causality.

Implementation checks. Use INTEGER PRIMARY KEY AUTOINCREMENT for each Id and INTEGER for foreign keys. Enable foreign-key enforcement on every connection. NOT NULL does not reject empty strings; validate these separately where empty values are not meaningful. Your DDL and report must agree about key uniqueness, NULL values, cardinalities, and delete behavior.